

جامعة عجمان
AJMAN UNIVERSITY

AIT111
Artificial Intelligence
Chapter 2

Overview

01 ⟶ Introduction to Machine Learning?

02 ⟶ Types of ML

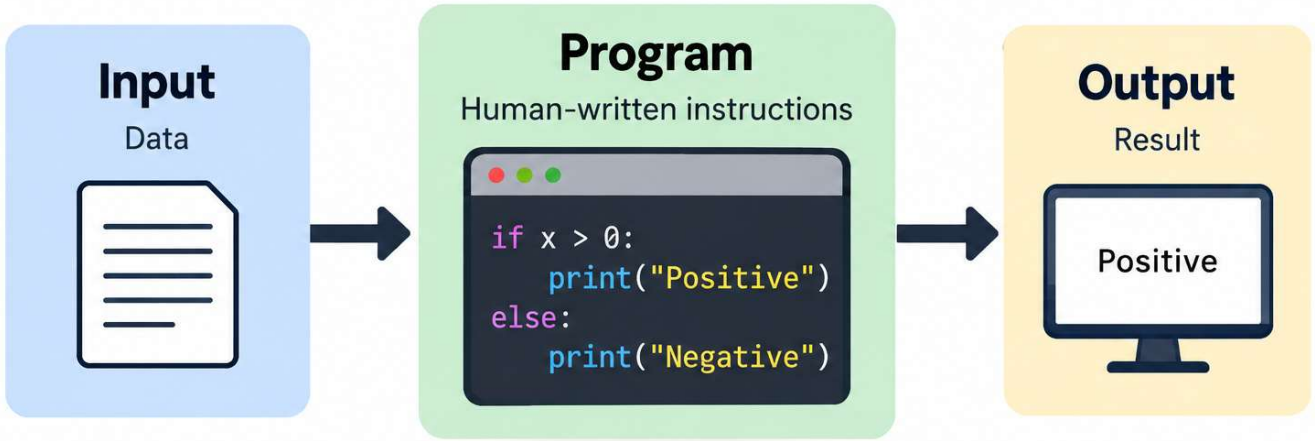
03 ⟶ Supervised Learning Methods

04 ⟶ Unsupervised Learning Methods

05 ⟶ Challenges & Considerations

What is Traditional Programming?

In traditional programming, a human writes explicit instructions (code) for the computer to solve a problem.



You tell the computer **exactly what to do**, step by step, and it follows those instructions to produce the output.

Applications of Traditional Programming

Traditional programming is widely used in real-world systems where the rules are clear and can be written as **explicit instructions.**

ATM Machine



Processes transactions, verifies accounts, and controls hardware using programmed instructions.

Coin-operated Shopping Box



Accepts coins, checks payment, and releases the selected item using predefined rules.

Lift (Elevator)



Controls movement between floors, handles requests, and ensures safety using programmed logic.

Calculator



Performs arithmetic operations using fixed rules and logical steps.

Car Access Control (Parking Gate)



Reads an access card, verifies permission, and opens the gate using programmed instructions.

01 →

What is Machine Learning?



Machine Learning: Training and Testing

1. Training (Learning from Data)

Labeled Training Data


Cat


Dog


Elephant

...

Learning Algorithm



Finds patterns in the data

Trained Model

The model learns from labeled examples to find patterns (features) in the data.

2. Testing (Making Predictions)

New (Unseen) Data







Trained Model

Predicted Output


Cat


Dog

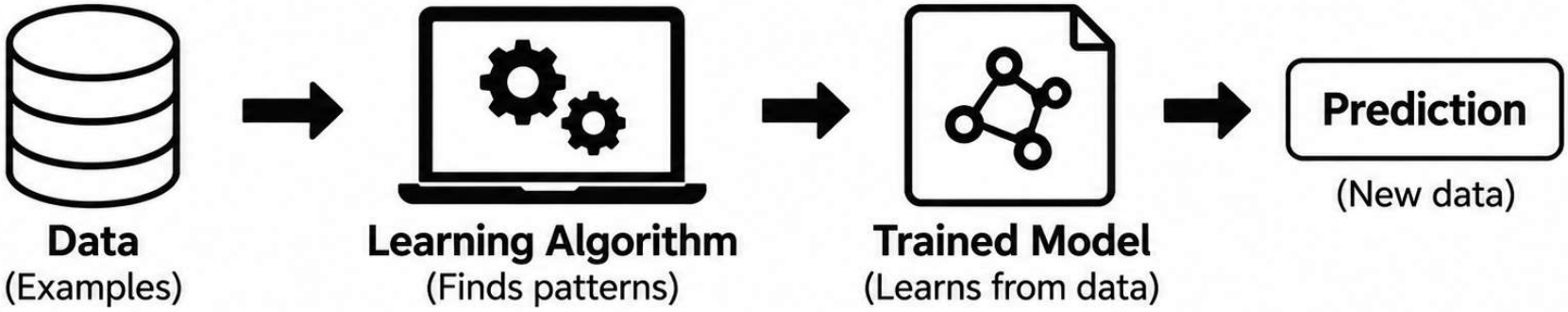

Elephant

The trained model is used to predict the label for new, unseen data.

Learn from examples → Make predictions on new data

What is Machine Learning (ML)?

Machine Learning (ML) is a branch of artificial intelligence that enables computers to **learn patterns from data** and make predictions or decisions **without being explicitly programmed.**

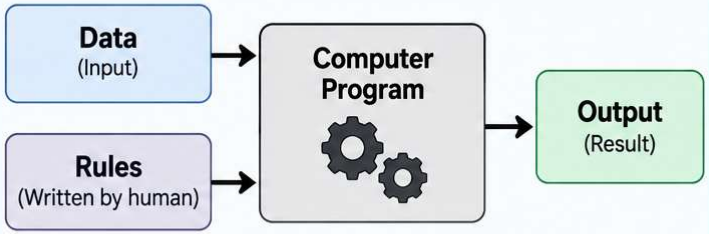


Learn from data → Improve performance → Make predictions

Traditional Programming vs Machine Learning

Traditional Programming (TP)

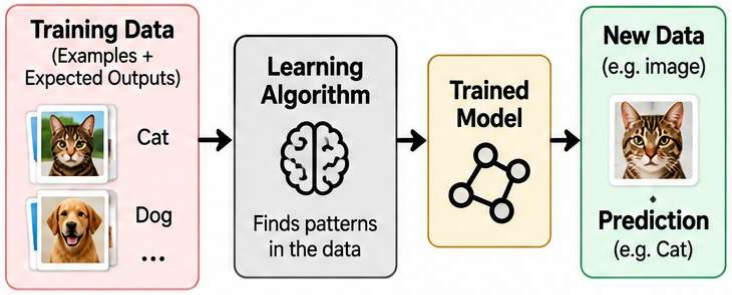
Rules are written by humans.



- Rules (logic) are explicitly written by humans.
- Data + Rules → Output
- Follows step-by-step instructions.
- Best for well-defined and simple problems.
- Does not learn automatically.
- Same input + same rules → same output.
- **Examples:** ATM, lift, calculator, parking gate, coin-operated machine.

Machine Learning (ML)

Rules are learned from data.



- Learns rules/patterns from data instead of being explicitly programmed.
- Data + Expected Outputs → Trained Model
- Can handle complex problems.
- Performance improves with more/better data.
- Can make predictions or decisions on new, unseen data.
- Same input may give different output if the model is updated.
- **Examples:** image recognition, spam detection, disease prediction, speech recognition.

Key Difference:

In Traditional Programming, humans write the rules. In Machine Learning, the computer learns the rules from data.

Labelled Image/Data




Unlabelled Data
Data without any labels or category information

Unsupervised Learning

Learn from unlabelled data (input only, no labels)

In unsupervised learning, we train a model using data without any labels.
The model finds hidden patterns, structure or groups in the data.

Input Data (Unlabelled)



No labels are provided
(just the input data)

Train

Unsupervised Learning Model

Find patterns

Output (Groups / Clusters)

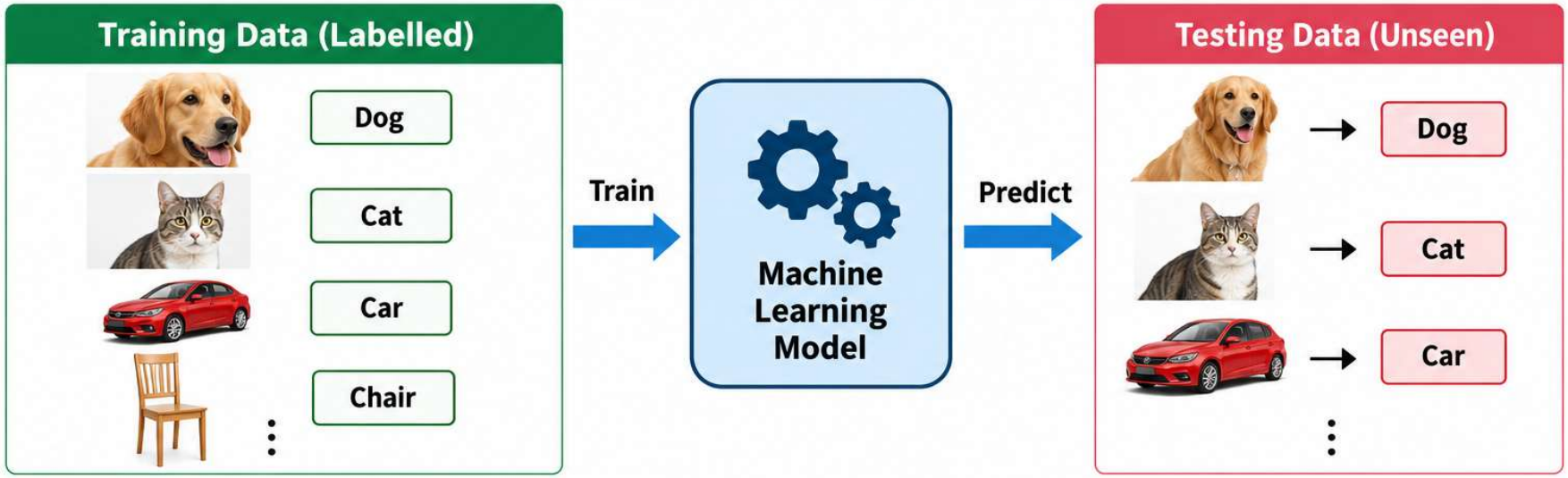


Similar items are grouped together
(no predefined labels)

Supervised Learning

Learn from labelled data (input + correct output)

In supervised learning, we train a model using labelled data, where each input is given with the correct output (label). The model learns the mapping from inputs to outputs and can then predict the label for new, unseen data.

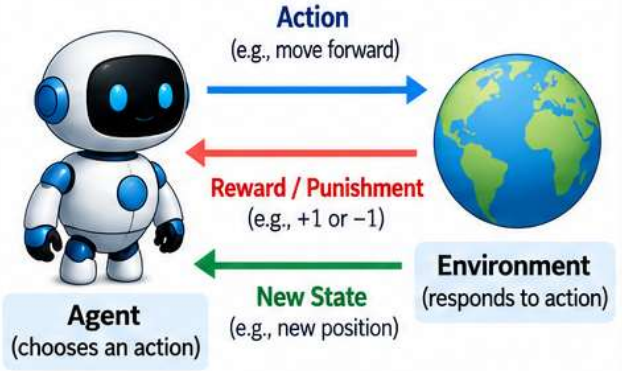


Reinforcement Learning

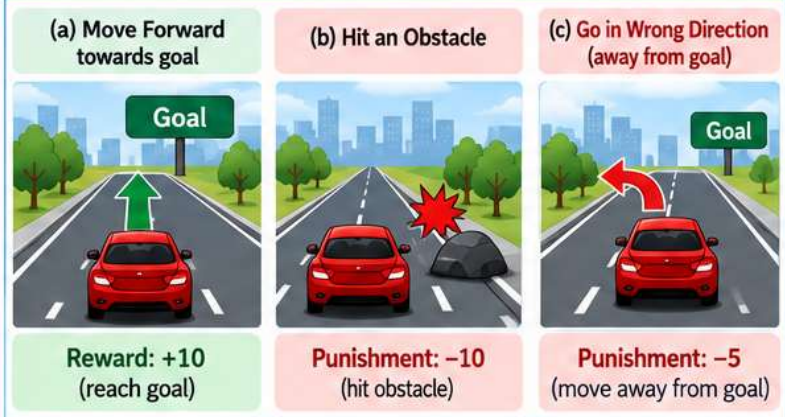
Learn by interacting with an environment (trial and error)

In reinforcement learning, an agent learns the best action by interacting with an environment.
The agent receives a reward (positive) or a punishment (negative) and improves its actions over time to maximize the total reward.

1. Basic Idea (Agent–Environment Interaction)

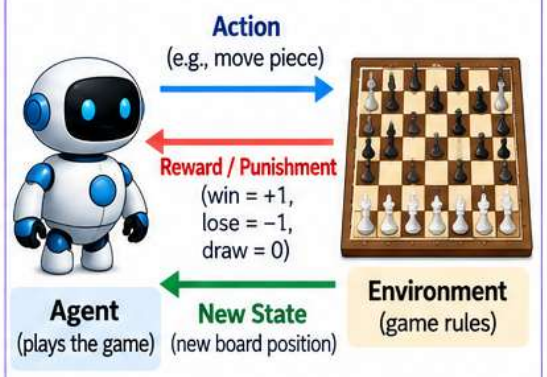


2. Example: Self-Driving Car (with Reward and Punishment)



Agent learns the best policy over many trials.

3. Example: Game Playing (e.g., Chess)



What is ML?

Definition of Machine Learning

Machine Learning is a subset of artificial intelligence that enables systems to learn from data and improve over time without explicit programming.

Why Machine Learning?

Traditional programming vs. Machine Learning

Examples of ML applications: Recommendation systems, image recognition, natural language processing.

Types of Machine Learning

Supervised Learning

Unsupervised Learning

Reinforcement Learning (brief mention)

Supervised vs Unsupervised Learning

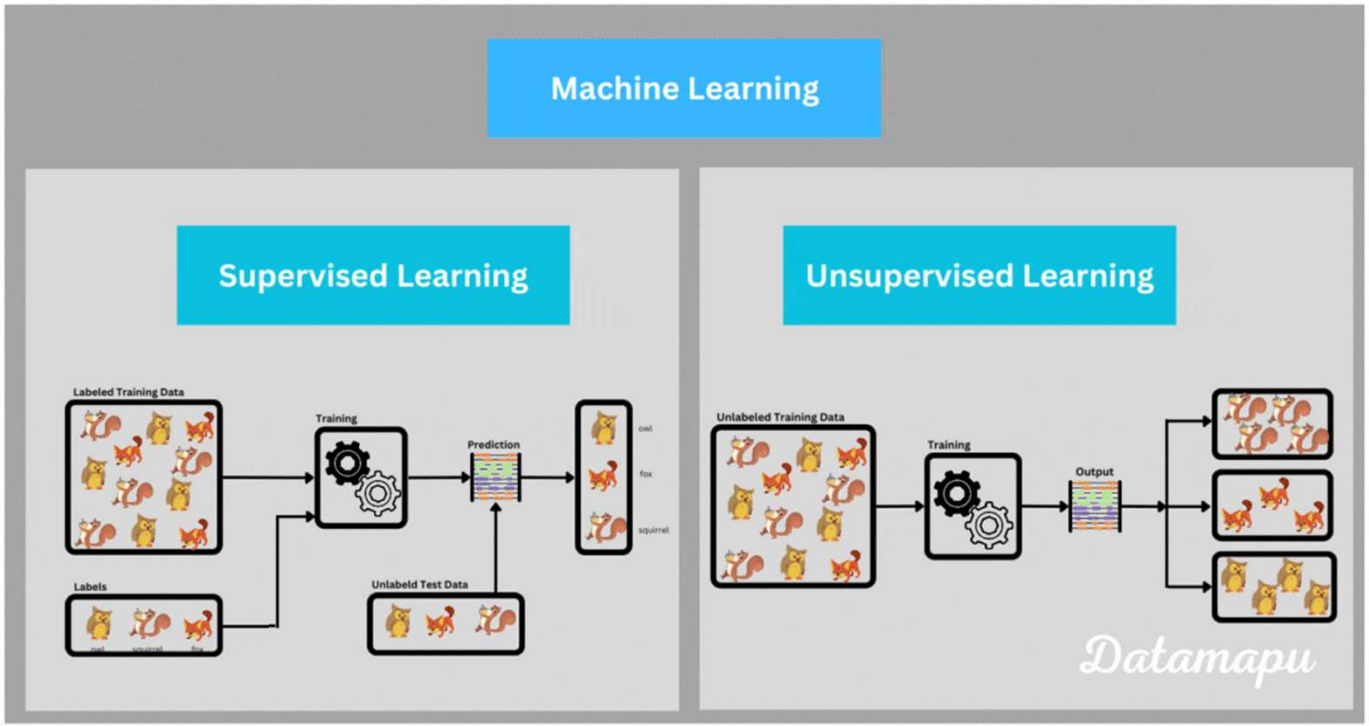
Supervised Learning

- Definition: Learning from labeled data (input-output pairs).
- Examples: Regression (predicting continuous values), Classification (predicting discrete values).
- Applications: Predicting house prices, spam email detection.

Unsupervised Learning

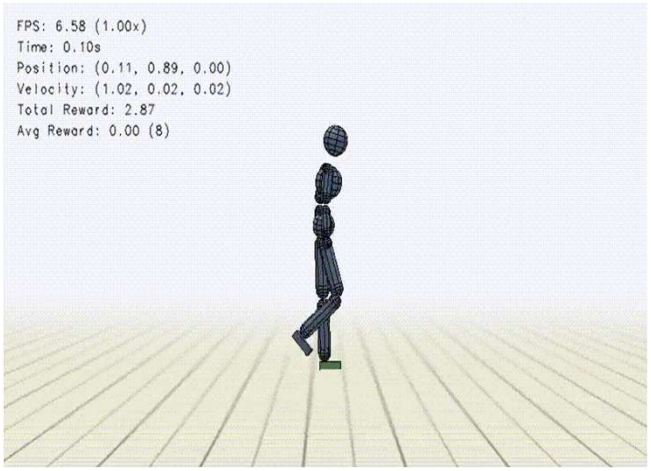
- Definition: Learning from unlabeled data.
- Examples: Clustering (grouping similar data points), Dimensionality Reduction (simplifying data without losing important features).
- Applications: Customer segmentation.

Supervised vs Unsupervised Learning



Reinforcement Learning

Reinforcement Learning (RL) is a type of machine learning where an agent learns to make decisions by interacting with an environment. It learns to achieve a goal through trial and error, receiving feedback in the form of rewards or penalties.



Key Components:

1. **Agent:** The learner or decision-maker that interacts with the environment.
2. **Environment:** The external system with which the agent interacts, where it performs actions and receives feedback.
3. **Actions:** Choices made by the agent that affect the environment.
4. **Rewards:** Feedback from the environment indicating the desirability of the agent's actions.
5. **Policy:** Strategy or method the agent uses to determine actions based on current state.

How It Works:

- **Trial and Error:** Agent learns by taking actions, receiving rewards, and updating its strategy (policy) to maximize future rewards.
- **Exploration vs. Exploitation:** Balancing between trying new actions (exploration) and choosing actions known to yield high rewards (exploitation).
- **Learning Process:** Typically involves iterative improvement of the policy through algorithms like Q-learning, SARSA, or deep reinforcement learning (DRL).
- Examples: AlphaGo, Chess, autonomous vehicles.

Supervised Learning Methods

Regression:

- Regression techniques are used when the target variable is continuous. The goal is to predict a continuous value, such as predicting house prices, stock prices, or temperature.
- Examples of regression algorithms include linear regression, polynomial regression, support vector regression (SVR), decision tree regression, and neural networks (for regression tasks).

Classification:

- Classification techniques are used when the target variable is categorical. The goal is to predict which category or class an observation belongs to.
- Examples of classification algorithms include logistic regression, decision trees, random forests, support vector machines (SVM), k-nearest neighbors (KNN), naive Bayes, and neural networks (for classification tasks).

Supervised Learning Methods

Regression



What will be the temperature tomorrow?

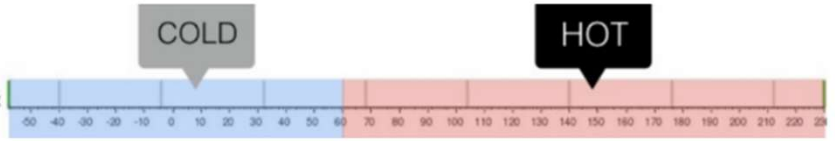


Fahrenheit

Classification

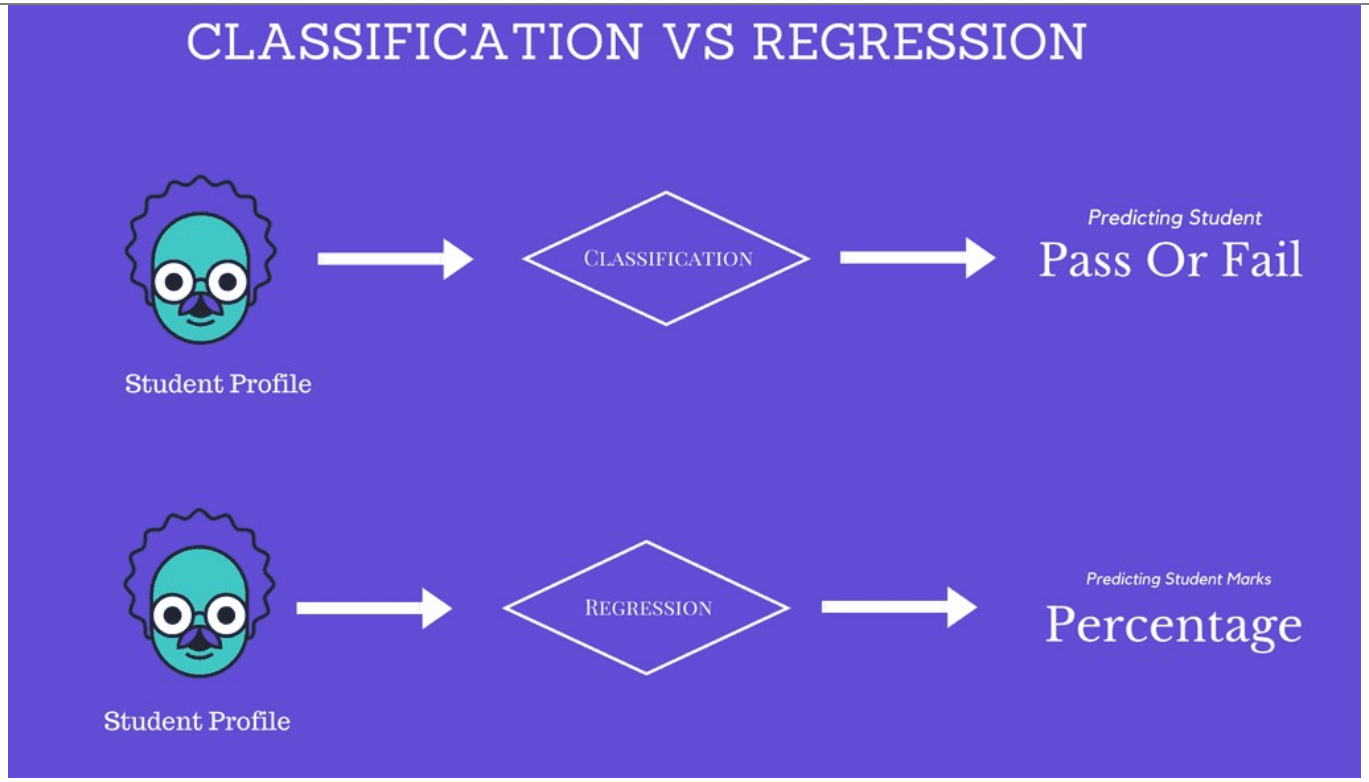


Will it be hot or cold tomorrow?



Fahrenheit

Supervised Learning Methods



Components of a Machine Learning Model

A simple supervised learning example

1. Input Features

Information given to the model



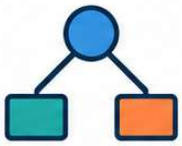
Example: house size and number of rooms

2. Model Structure

The mathematical form used to make predictions

$$y = w_1x_1 + w_2x_2 + b$$

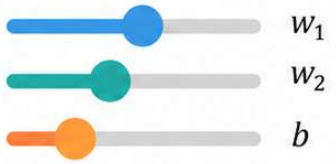
or



Example: a linear equation or decision tree

3. Learnable Parameters

Values adjusted during training



Example: weights and bias

4. Loss Function

Measures prediction error



Compares predicted and actual prices

5. Learning Algorithm

Updates parameters to reduce error



Example: gradient descent

6. Prediction

The output produced by the model



Example: estimated house price

Training uses examples with known answers. The trained model predicts answers for new inputs.

Training, Validation and Test Sets

Three separate parts of a machine learning dataset

1. Training Set

LEARN



The model learns patterns and parameters.

Like studying lessons

2. Validation Set

TUNE



Used to choose model settings and select the best model.

Like a practice exam

3. Test Set

EVALUATE



Used to measure final performance on unseen data.

Like the final exam

Keep the test set untouched during training and model selection.

Validation guides model development; the test set checks the final result.

Data Organization in Machine Learning

Example: predicting house prices

	Input features (X)		Target (y)
	House	Size (m²)	Rooms	Price (AED)
Rows = samples	A	100	2	500,000
	B	150	3	750,000
	C	200	4	1,000,000

1. Training Set

Used to learn model parameters



Example: 70% of the houses

2. Validation Set

Used to tune model settings



Example: 15% of the houses

3. Test Set

Used for final evaluation on unseen data



Example: 15% of the houses

Keep the sets separate. Fit preprocessing on the training set only.

Problem:

A dataset contains information about **3,800 houses**. It is divided into **70% for training**, **10% for validation**, and **20% for testing**.

Calculate the number of houses in each set.

Dataset Split: 3,800 Houses

70% Training • 10% Validation • 20% Testing

Training Set



70%

3,800 × 70 ÷ 100

= 2,660 houses

Validation Set



10%

3,800 × 10 ÷ 100

= 380 houses

Testing Set



20%

3,800 × 20 ÷ 100

= 760 houses

Check: 2,660 + 380 + 760 = 3,800 houses

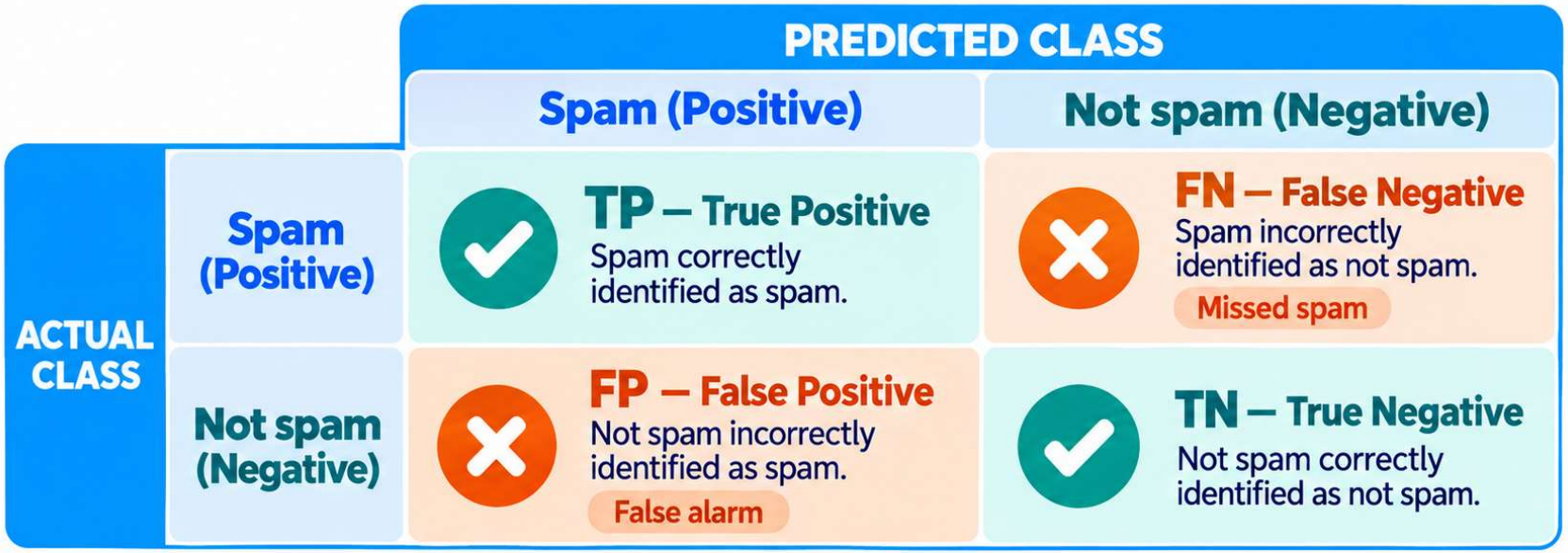
Confusion Matrix – Basics

- A confusion matrix is a table that shows how well a classification model performs.
- It compares the actual values with the predicted values.
- It has 4 main parts:
  - True Positive (TP): Correctly predicted positives.
  - True Negative (TN): Correctly predicted negatives.
  - False Positive (FP): Predicted positive but actually negative.
  - False Negative (FN): Predicted negative but actually positive.

Binary Confusion Matrix

Example: detecting spam emails

Positive = Spam | Negative = Not spam



Correct predictions: TP + TN

Incorrect predictions: FP + FN

True = correct prediction. False = incorrect prediction.

Problem

A classifier checks **100 emails**: **40 are spam** and **60 are not spam**. It correctly identifies **32 spam emails** and **48 non-spam emails**.

Taking **spam as the positive class**, find **TP, FN, FP, and TN**, draw the **confusion matrix**, and calculate the classifier's **accuracy**.

Confusion Matrix: Solution

100 emails: 40 spam and 60 non-spam
Correctly identified: 32 spam and 48 non-spam
Positive = Spam

		PREDICTED CLASS	
		Spam (Positive)	**Not spam (Negative)**
ACTUAL CLASS	**Spam (Positive)**	**TP = 32** ✔ Correctly identified spam	**FN = 8** ✖ 40 – 32 = 8 missed spam
	Not spam (Negative)	**FP = 12** ✖ 60 – 48 = 12 false alarms	**TN = 48** ✔ Correctly identified non-spam

Accuracy = (TP + TN) ÷ Total × 100%
Accuracy = (32 + 48) ÷ 100 × 100% = 80%

Check: 32 + 8 + 12 + 48 = 100 emails

Precision, Recall & F1-Score

Three key metrics for evaluating classification models

PRECISION

Of all predicted positives, how many are correct?

$$\text{Precision} = \frac{TP}{TP + FP}$$



Fewer false alarms

RECALL (SENSITIVITY)

Of all actual positives, how many are detected?

$$\text{Recall} = \frac{TP}{TP + FN}$$



Fewer missed positives

F1-SCORE

Harmonic mean of precision and recall

$$F1 = \frac{2PR}{P + R}$$

$$F1 = \frac{2TP}{2TP + FP + FN}$$



High when both are high

Find Precision, Recall, and F1?

		PREDICTED CLASS	
		Spam (Positive)	Not spam (Negative)
ACTUAL CLASS	Spam (Positive)	**TP = 32** ✔ Correctly identified spam	**FN = 8** ✘ 40 – 32 = 8 missed spam
	Not spam (Negative)	**FP = 12** ✘ 60 – 48 = 12 false alarms	**TN = 48** ✔ Correctly identified non-spam

Precision, Recall & F1-Score – Solution

		PREDICTED CLASS: Spam (Positive)	PREDICTED CLASS: Not spam (Negative)
ACTUAL CLASS	Spam (Positive)	**TP = 32** ✔ Correctly identified spam	**FN = 8** ✘ 40 – 32 = 8 missed spam
	Not spam (Negative)	**FP = 12** ✘ 60 – 48 = 12 false alarms	**TN = 48** ✔ Correctly identified non-spam

Precision

Of all emails predicted as spam, how many are actually spam?

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{32}{32 + 12} = \mathbf{72.73\%}$$

Recall

Of all actual spam emails, how many did we correctly identify?

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{32}{32 + 8} = \mathbf{80.00\%}$$

F1-Score

Harmonic mean of precision and recall (balances both).

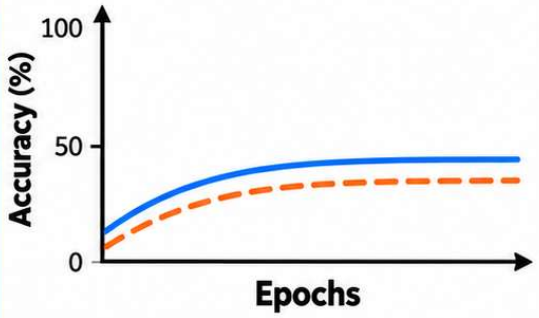
$$F1 = \frac{2TP}{2TP + FP + FN} = \frac{64}{64 + 12 + 8} = \mathbf{76.19\%}$$

Training and Validation Accuracy Curves

Underfitting • Good Fit • Overfitting

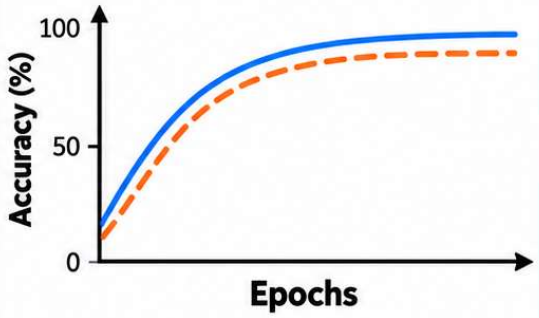
Training accuracy — — — Validation accuracy

Underfitting



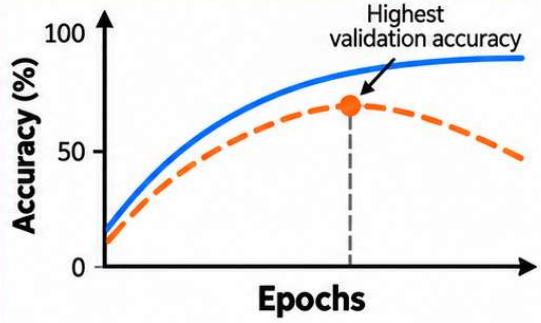
Both accuracies remain low.
The model has not learned enough.

Good Fit (Best Fit)



Both accuracies are high and close.
The model generalizes well.

Overfitting



Training accuracy rises; validation accuracy falls.
Performance on unseen data worsens.

When selecting by accuracy, choose the highest validation accuracy.

Illustrative curves — actual training curves may vary.

Evaluation Metrics

Purpose: Assessing model performance to gauge effectiveness and reliability.

Optimization: Metrics guide model tuning and selection to meet specific goals.

Comparison: Facilitate comparison between different models or algorithms

Regression Metrics

1. Mean Squared Error (MSE)

Definition: Average of the squared differences between predicted and actual values.

Use: Measures overall model accuracy, penalizes larger errors.

Formula: $\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$

2. R-squared (Coefficient of determination)

Definition: Proportion of the variance in the dependent variable that is predictable from the independent variables.

Interpretation: Higher R-squared values indicate better model fit.

Range: $0 \leq R^2 \leq 1$, where 1 indicates perfect prediction.

Evaluation Metrics

Classification Metrics

1. Accuracy

Definition: Ratio of correctly predicted observations to the total observations.

Use: Basic measure of overall model performance when classes are balanced.

Formula:

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

2. Precision, Recall, F1-score

Precision: Proportion of true positive predictions among positive predictions.

Recall (Sensitivity): Proportion of true positives correctly identified.

F1-score: Harmonic mean of precision and recall, balances both metrics.

Use: Useful for imbalanced class distributions.

Formulas:

- Precision: $\text{Precision} = \frac{TP}{TP+FP}$
- Recall: $\text{Recall} = \frac{TP}{TP+FN}$
- F1-score: $F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$

Challenges & Considerations

1. **Overfitting:** Occurs when a model learns too much from the training data, capturing noise or random fluctuations rather than the underlying pattern. This leads to poor performance on unseen data.

- **Mitigation Techniques:**
  a) **Cross-validation:** Use techniques like k-fold cross-validation to assess model performance on multiple subsets of the data.
  b) **Regularization:** Apply techniques like L1 (Lasso) or L2 (Ridge) regularization to penalize complex models and prevent overfitting.
  c) **Early Stopping:** Stop training when performance on a validation set starts to degrade, preventing the model from learning noise.
  d) **Data Augmentation:** Increase the size and diversity of the training data to expose the model to a broader range of scenarios.

Challenges & Considerations

2. **Underfitting:** Occurs when a model is too simplistic to capture the underlying pattern of the data, resulting in high bias and poor performance on both training and test data.

- **Mitigation Techniques:**
    a) **Increase Model Complexity:** Use more complex models such as deep neural networks or ensemble methods that can capture more intricate patterns in the data.
    b) **Feature Engineering:** Introduce new features or transform existing features to provide more information to the model.
    c) **Reduce Regularization:** If regularization parameters are too high, it can lead to underfitting. Adjust regularization parameters based on model performance.

Challenges & Considerations

3. **Bias-Variance Tradeoff:**

Bias: Error introduced by approximating a real-world problem with a simplified model.

Variance: Sensitivity of the model to variations in the training data.

Tradeoff: As you decrease bias (by increasing model complexity), you typically increase variance and vice versa.

- **Mitigation Techniques:**
  a) **Regularization:** Helps in reducing variance by penalizing the complexity of the model.
  b) **Ensemble Methods:** Combine predictions from multiple models to reduce variance (e.g., bagging, boosting, stacking).
  c) **Cross-validation:** Helps in assessing both bias and variance, allowing for model selection that balances these aspects effectively.
  d) **Model Selection:** Choose models that strike a good balance between bias and variance based on the problem at hand and available data.

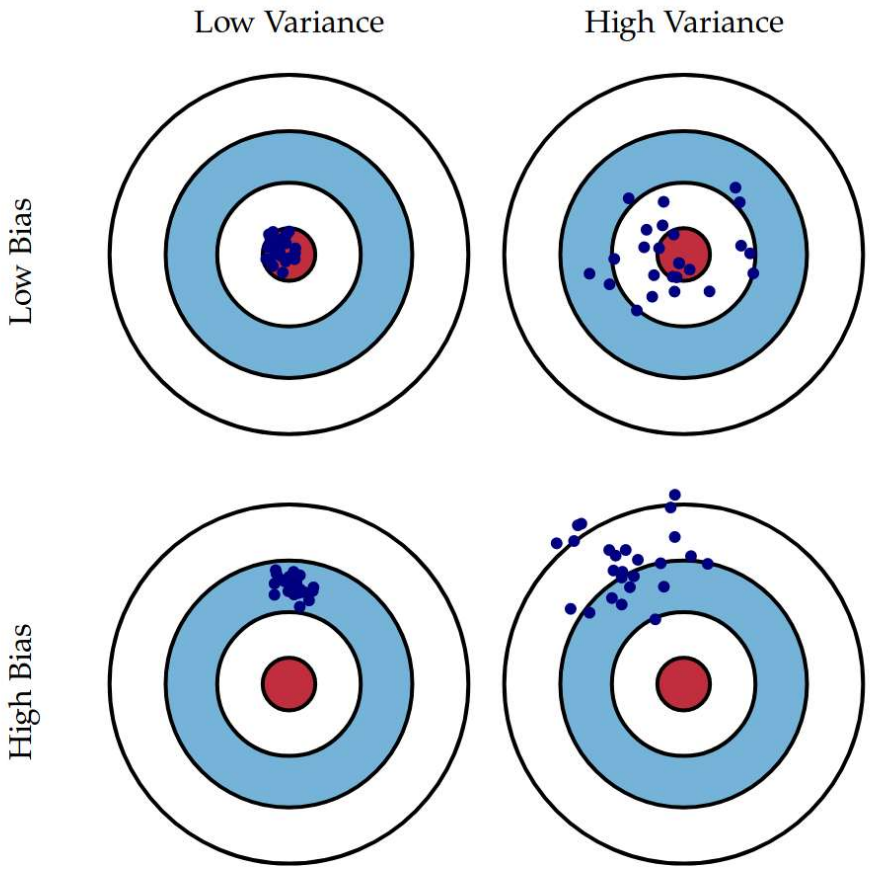


Fig. 1 Graphical illustration of bias and variance.

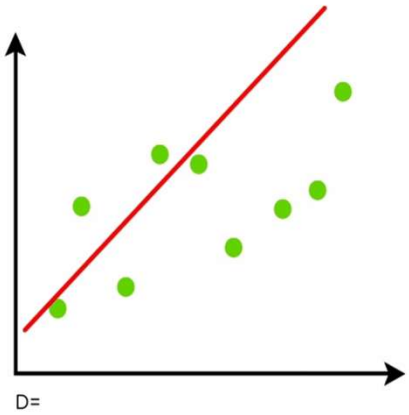
Linear Regressor

Linear regression is a statistical technique used to model the **linear relationship** between a dependent variable and one or more independent variables. The dependent variable is the variable that is being predicted, while the independent variables are the variables that are used to predict the dependent variable. Linear regression is used to find the **equation of a line** that best fits a set of data points.

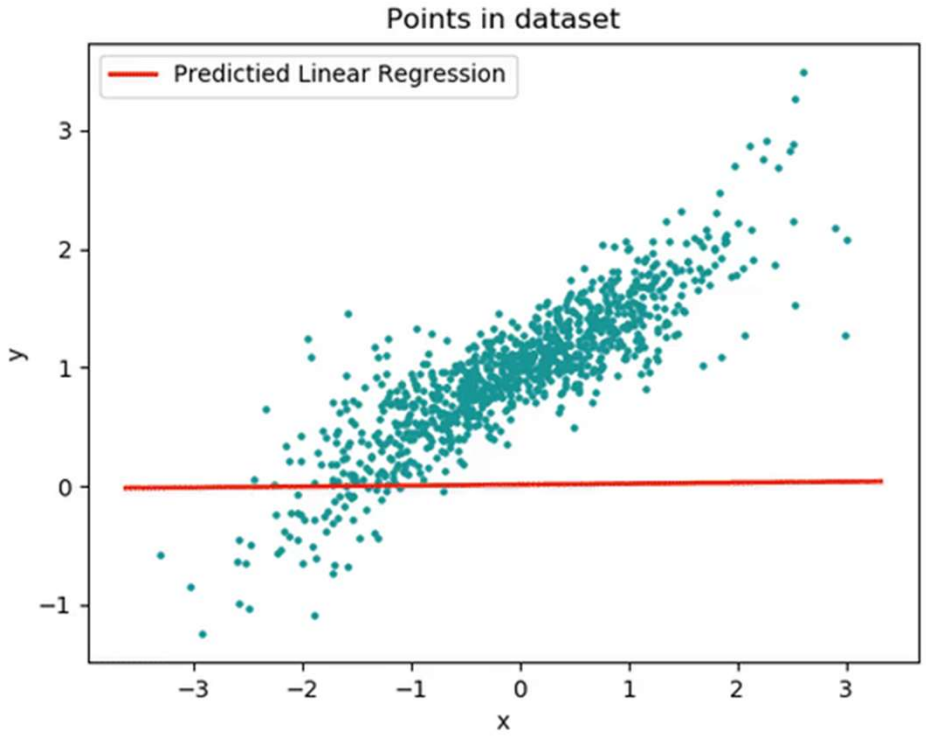
The line of best fit is the line for which the error between the predicted values and the observed values is minimum.

The red line is the regression line, the green dots are the observed data value, and d1,d2,d3,….are the error values(y actual — y pred).

We rotate the regression line (change the slope of the line) and translate the regression line vertically (change the y-intercept) until the error is minimum.



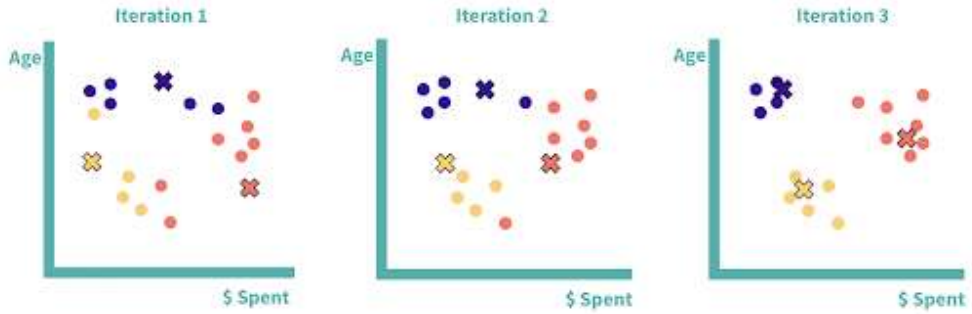
Linear Regressor

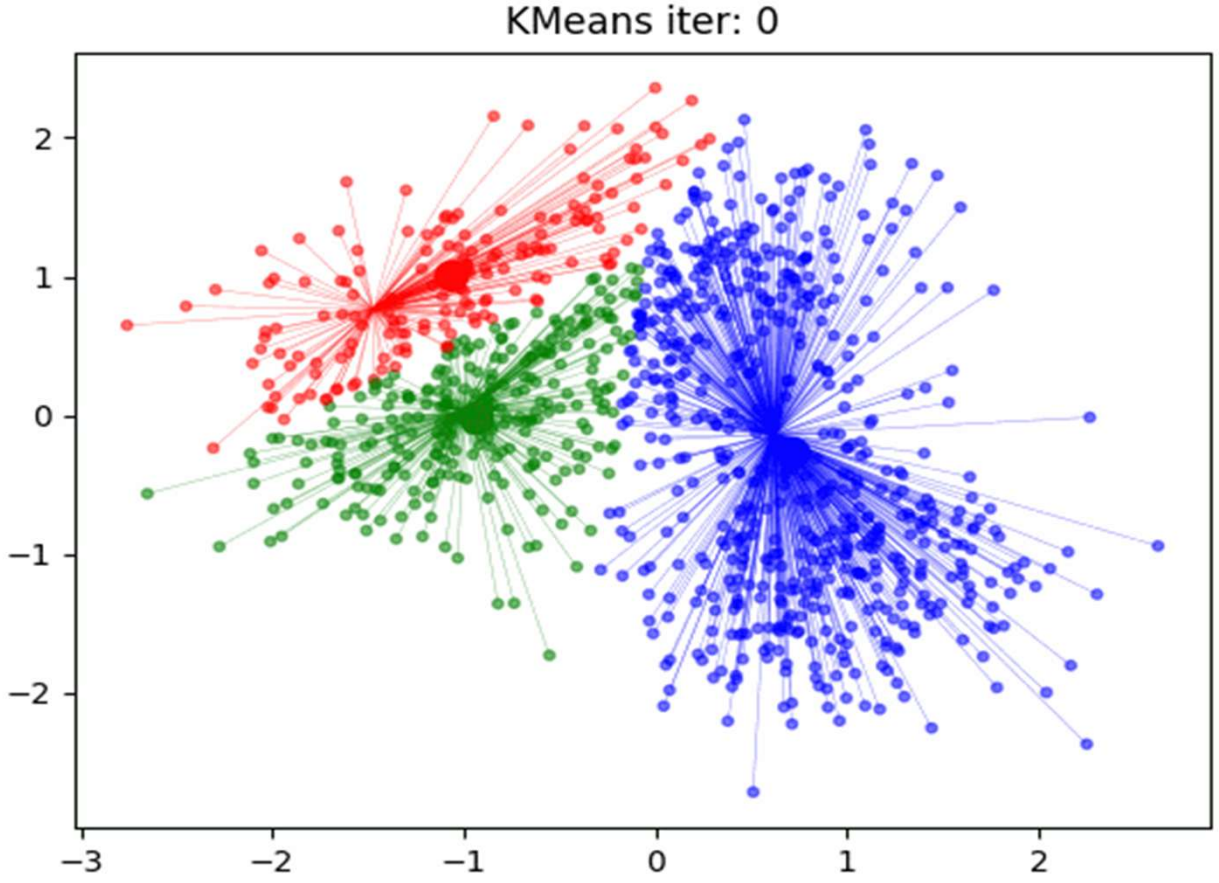


Clustering

K-means clustering is a method of separating data points into several similar groups, or "clusters," characterized by their midpoints, which we call **centroids**.

1. Select K, the number of clusters you want to identify. Let's select K=3.
2. Randomly generate K (three) new points on your chart. These will be the centroids of the initial clusters.
3. Measure the distance between each data point and each centroid and assign each data point to its closest centroid and the corresponding cluster.
4. Recalculate the midpoint (centroid) of each cluster.
5. Repeat steps three and four to reassign data points to clusters based on the new centroid locations. Stop when either:
   a. The centroids have been stabilized; after computing the centroid of a cluster, no data points are reassigned.
   b. The predefined maximum number of iterations has been achieved.




KMeans iter: 0
2
1
0
−1
−2
−3
−2
−1
0
1
2

Q&A